John Botonakis & Adrian Rincon Jimenez
CS-410
04/13/25

Explanation of the Program

When running the program, you are executing the main method inside the DiningPhilosophers class, which acts as the central entry point. It begins by creating an array of five Chopstick instances and five Philosopher threads, with each philosopher sharing the chopsticks between them in a circular arrangement. These chopsticks are protected using ReentrantLock to ensure mutual exclusion — no two philosophers can hold the same chopstick at the same time. The philosophers are then submitted to a fixed thread pool, where each begins alternating between thinking, getting hungry, and eating. Before a philosopher can eat, they must acquire both their left and right chopsticks. To avoid deadlock, philosopher #4 (the last one) picks up the right chopstick first instead of the left, creating asymmetry and breaking circular wait conditions. Each philosopher eats five times before they declare themselves full and exit the simulation. Meanwhile, the main thread continuously checks if all philosophers are full, and once they are, it gracefully shuts everything down, prints out the number of times each philosopher ate, and ends the program.

Meeting the Requirements

We believe this program meets all the specified requirements for the Dining Philosophers lab. Starting out, each philosopher is its own separate thread, implemented via the Runnable interface and executed within a fixed thread pool. Chopsticks are shared resources, protected using ReentrantLock objects with fairness enabled to reduce starvation. The main thread keeps track of when all philosophers have completed their meals, using the isFull() check on each Philosopher instance. To prevent deadlock, the fifth philosopher is assigned reversed chopstick-pickup logic, which guarantees that the system never enters a circular wait state. All thread operations are safely synchronized, and the program ends only after all work is done, with printed output confirming that each philosopher ate exactly five times. The implementation demonstrates an understanding of Java concurrency tools like ExecutorService, ReentrantLock, AtomicInteger, and volatile flags, and successfully avoids common pitfalls like deadlock and race conditions.

John Botonakis & Adrian Rincon Jimenez
CS-410
04/13/25

Challenges

Some of the challenges we've faced include configuring the locks for the chopsticks and ensuring each philosopher had enough to eat. The chopsticks continuously gave us issues where all the philosophers would grab onto both chopsticks and never release them, or fail to get any of them entirely, leading to a deadlock. While it was kinda cool to see a demonstrable Deadlock in action, it was not so cool to diagnose and fix it. Ensuring all philosophers ate was our other big issue. Without the correct timing control and really racking our brains as to what was "fair", some philosophers thought until they died, and some would just eat way more than others. We solved it by introducing the randomized delays and by making the locks to access the queue better.